

COMPLETE ECOSYSTEM ARCHITECTURE

How Inaya Actually Works

Every contract, app, backend, and data flow across the ecosystem — written from the real code, not the pitch. For the founding team.

Document INAYA-ARCH-2026-V1 · Classification Internal · August 2026

SECTION 01

How To Read This Document

Inaya is one product with two connected engines: a decentralized storage/DePIN protocol (on-chain contracts, node operators, client-side encryption) and a set of applications built on top of it (the web dApp, Business Workspace SaaS, mobile app, two desktop apps, and three AI assistants). This document maps every piece, what it actually does, and how it connects to everything else.

Conventions used throughout:

- Everything here is on BNB Chain Testnet unless stated otherwise — nothing described is live on mainnet yet.
- Contract addresses given are real, deployed, and verified on BscScan as of this writing.
- Where a feature is deliberately unbuilt or deferred, it's labeled as such rather than glossed over — this is a working reference, not a pitch document.
- "Backend" always means the Next.js app at inaya-network-dapp — every mobile screen, desktop app, and the web dApp itself all call the same backend, there is no separate API server.

SECTION 02

The System At A Glance

REPO / APP	WHAT IT IS	RUNS ON
inaya-network-dapp	The backend (Next.js API routes + MongoDB) AND the main web dApp AND the Business Workspace web UI — one Next.js app serving all three.	Vercel, Node.js
contracts/ (repo root)	Solidity source for the protocol's own contracts (Staking, Vault, Escrow, Node Registry, Proof Registry, Security Layer). Deployed to BSC Testnet.	Solidity 0.8.20, Hardhat
custody-sdk	Client-side encryption/sharding library ("InayaKernel") — the code that actually encrypts and splits a file before it ever leaves a user's device.	JS, @noble crypto primitives, ethers v6
custody-sdk/packages/node-daemon	CLI a storage-node operator runs — registers on-chain, sends heartbeat telemetry. Published to npm.	Node.js CLI
inaya-mobile	React Native / Expo mobile app — a superset of the web dApp's consumer features plus Business Workspace, Learn, and Security.	Expo, React Native
inaya-desktop	Tauri (Rust) native wrapper around the Business Workspace web app.	Tauri, WebView2 (Win) / WebKitGTK (Linux)
inaya-dapp-desktop	Tauri native wrapper around the main dApp (faucet, staking, etc).	Tauri, same engines as above

None of the desktop or mobile apps run their own backend logic beyond thin native chrome — every one of them calls the exact same inaya-network-dapp API routes the website does.

SECTION 03

The Two Engines

ENGINE 1 — DEPIN PROTOCOL

The foundation: users encrypt files client-side, shard them, and anchor ownership + integrity proofs on-chain. Independent node operators register capacity, earn \$INAYA/USDT for storage and threat-reporting, and settle through timelocked, oracle-attested (not cryptographic) verification. This is the part that makes Inaya a real DePIN, not just a SaaS product with a token bolted on.

ENGINE 2 — APPLICATIONS

Everything a user actually touches: the web dApp, Business Workspace (SaaS document management for companies that will never think about blockchain), mobile app, two desktop wrappers, a public Security transparency page, an investor Data Room, and three Gemini-powered AI assistants. All of it sits on top of Engine 1's infrastructure, but most of it is reachable — and usable — without a wallet.

Why this split matters. It's the reason a non-crypto business customer can use the Business Workspace with just an email address while a node operator, three layers down, is settling USDT through a timelocked escrow contract — the same infrastructure serves both, but almost nothing about Engine 1 leaks into Engine 2's user experience.

SECTION 04

On-Chain Layer — Contracts

10 contracts deployed and verified on BSC Testnet, split across two families that don't share any direct dependency on each other.

Core Protocol (6 contracts)

Custody, payments, staking, and node settlement.

CONTRACT	ADDRESS	JOB
InayaCustody ("Core Custody Contract")	0x7F5E6c...5a888	Asset-ownership ledger. <code>batchRegisterAssets()</code> writes <code>fileHash</code> → {owner, shard CIDs, size, timestamp}. This is the single source of truth for "who owns this file." Note: its Solidity source isn't in this repo — only its interface (<code>INayaCustody</code>) is, referenced from <code>InayaProofRegistry</code> . It's deployed and live, just not tracked as source here yet.
Mock USDT (mUSDT)	0x6f16E2...45D	Testnet stand-in for USDT — every fee, stake, escrow, and invoice in the protocol is denominated in this token until mainnet.
\$INAYA Token	0x3966a3...e94e	The real ERC-20 utility/governance token. Fixed supply, 30,000,000 tokens (per tokenomics).
InayaNodeRegistry	0xd12a38...FD881	Node operator registry + timelocked commission settlement. <code>registerNode(capacityGB)</code> , tiered commission (30/40/50% by Entry/Mid/Enterprise), verifier-attested metrics, 36-hour settlement delay before anyone can call <code>releaseSettlement()</code> .
RevenueRouter	0x76B0d4...3256	<code>processCorporateInvoice(usdtAmount)</code> — entry point for Corporate Reserve plan purchases. Source not in this repo, referenced by address from the Stripe webhook and <code>page.js</code> .
InayaProofRegistry	0xEdF431...CEBcB	Merkle-root storage per asset + chunk-proof verification (<code>registerMerkleRoot</code> , <code>verifyChunkProof</code>). Reads <code>InayaCustody</code> to confirm the caller owns the asset before letting them register a root.

Also in contracts/ but not part of the six above: `InayaStaking` (Synthetix-style staking-rewards pool, 0/30/90-day lock tiers at 1.00x/1.25x/1.50x multiplier), `InayaEgressTimelockVault` (holds egress-fee \$INAYA in 180-day epochs, forwards it to Staking; also sweeps USDT storage revenue to the operational treasury), `InayaCorporateEscrow` (drips a node operator's 39% COGS share of a corporate invoice over 12 monthly releases), and `MockINAYA` (test-only ERC-20 stand-in for \$INAYA, not deployed to testnet under the real token address). All four are deployed via their own scripts/`deploy-*.cjs`; addresses live in `.env.local` and are read by `page.js` as `stakingContractAddress` / etc. — not restated here to avoid publishing addresses that weren't independently re-verified for this document.

Security Layer (4 contracts)

Decentralized threat intelligence — fully separate dependency graph from the core protocol above. Full detail in Section 09.

CONTRACT	ADDRESS	JOB
InayaThreatRegistry	0xb374dE...8adD	Threat status ledger, updated only by InayaThreatReporter.
InayaThreatReporter	0xe22EbA...6450B	confirmThreat() — relayer-only, called once a threat crosses the confidence threshold off-chain. Writes into ThreatRegistry.
InayaNodeReputation	0xD25836...9E27A	Periodic on-chain checkpoints of node reputation scores (real-time reputation itself lives off-chain).
InayaSecurityPolicy	0xCE1646...1976a	Versioned policy hash + URI, so a client can verify a cached policy offline without a live RPC call.

Trust-model honesty, on the record. InayaNodeRegistry's own header comment is explicit: node metrics are "coordinator-verified," not cryptographically proven — an authorized verifier wallet attests uptime/capacity off-chain. InayaProofRegistry's chunk-proof verification is currently onlyOwner (the backend checks proofs itself); the contract's own comment lays out a documented future path to make this permissionless and stake-slashing-backed. Neither is hidden — both are stated in the contract source itself.

SECTION 05

Client-Side Encryption & Storage — the custody-sdk

"InayaKernel" (custody-sdk) is the code that actually makes the zero-knowledge claim true. Published as @inaya-network/custody-sdk, used identically by the web dApp and the mobile app.

1. **Key derivation.** `deriveVaultKey()` runs PBKDF2 (default SHA-256, 100,000 iterations) over the user's passkey plus a fresh random 16-byte salt, producing a 32-byte AES key. The passkey itself never leaves the device and is never transmitted anywhere.
2. **Encrypt + shard.** `disperseAndSlice()` reads the file, base64-encodes it, AES-GCM-256 encrypts it with a fresh random IV, prepends salt+IV to the ciphertext, then splits that resulting base64 string at its exact character midpoint into two shards (`shardAlpha` / `shardBeta`). This is literal binary bisection — neither half means anything on its own, but it is not erasure coding or Reed-Solomon.
3. **Pin to IPFS.** The SDK itself does NOT upload anywhere — that's the calling app's job. `page.js` pins each shard to Pinata separately and gets back two CIDs (`cidAlpha`, `cidBeta`).
4. **Anchor on-chain.** `anchorToLedger()` calls `InayaCustody.batchRegisterAssets(fileHashes, fileSizes, shardACIDs, shardBCIDs)` — the CIDs (pointers, not the shard bytes themselves) are what actually gets written on-chain.
5. **Reconstruct + decrypt.** `retrieveAndReconstruct()` reads `InayaCustody.assets(fileHash)` to get both CIDs, fetches both shards from the Pinata gateway in parallel, concatenates them back into the original base64 string, splits out salt/IV/ciphertext, re-derives the AES key from the user's passkey, and decrypts.

Zero-knowledge, verified against the actual code — not assumed. Every encryption, key-derivation, and decryption step happens locally using only inputs the device already has (passkey, file bytes). No network call anywhere in custody-sdk's source carries plaintext or the derived key. The only backend/chain-touching write is a file hash, size, and two CID strings — never file content, never the key.

A second, separate feature — sharing without exchanging the raw passkey:

- `encryptForPublicKey` / `decryptWithSecretKey` / `deriveEncryptionKeypairFromSignature` — re-wraps the passkey itself (not the file) using X25519 ECDH + HKDF-SHA256 + XChaCha20-Poly1305 sealed-box encryption, so a recipient can unlock a shared vault entry without the passkey ever being transmitted in the clear.

Known gap worth flagging: the SDK's own bundled example (`examples/node-script.mjs`) skips the Pinata step and passes raw ciphertext directly into `anchorToLedger`'s CID parameters — it only "works" because the example never calls `retrieveAndReconstruct` against real IPFS. The production code path in `page.js` does the Pinata step correctly; the example just isn't a faithful end-to-end demo.

SECTION 06

Node Operator Layer

What someone actually runs to operate an Inaya storage node today, and what it does and does not do.

NODE-DAEMON (PUBLISHED NPM CLI)

Commands: login, register <capacityGB>, start, report <indicator> (Security Layer), service install/uninstall.
Wallet key is PBKDF2+AES-GCM encrypted at rest locally (~/.inaya/node-daemon/config.json). register() calls InayaNodeRegistry.registerNode() on-chain AND separately POSTs to the backend's /api/nodes/register for capacity bookkeeping. start() runs a 5-minute heartbeat loop POSTing telemetry to /api/nodes/heartbeat.

WHAT IT DELIBERATELY DOES NOT DO

It does not store or serve shards, does not host any file content, and does not execute settlement/payout logic — the daemon's own code comments are explicit that commission settlement is verifier- and relayer-driven, not something the daemon touches. Its on-chain surface is exactly two calls: registerNode and a read of nodes(address). It is an identity + registration + heartbeat agent, nothing more, today.

Settlement flow (InayaNodeRegistry, off-daemon):

- An authorized verifier wallet calls queueSettlement / queueSettlementsBatch — computes commission by tier, does not move funds yet.
- After a mandatory 36-hour delay, releaseSettlement is publicly callable by anyone (deliberately — a single compromised verifier key can queue a bad settlement but can never immediately drain funds).
- Corporate invoice revenue flows separately: RevenueRouter.processCorporateInvoice() → InayaCorporateEscrow.createEscrow() escrows a node operator's 39% COGS share and drips it out over 12 monthly releases via releaseMonthlyPayout(), callable by anyone once due.

SECTION 07

Web dApp — Protocol Surface

src/app/page.js — one large tabbed SPA. Every tab below is wallet-driven except Referrals (email + KYC only, no wallet).

TAB	WHAT IT DOES
Network Home	Landing panel only — status tiles, CTA into Sovereign Vault, cross-promo into Business Workspace / desktop downloads. No contract or API calls of its own.
Faucet	POSTs to /api/faucet; the route dispenses test \$INAYA/USDT server-side from a custodial faucet wallet. Self-limits by checking the wallet's existing balance rather than a visible cooldown timer.
Sovereign Vault	The core product. Upload: encrypt+shard client-side (custody-sdk) → pin to Pinata → InayaCustody.batchRegisterAssets() → InayaProofRegistry.registerMerkleRoot() per file. Download: reverse of the same path. Also has a no-wallet path for Stripe card customers (upload happens locally, on-chain registration happens server-side post-payment).
Corporate Reserve (sidebar panel + Business Model tab)	The sidebar tier selector (250/500/1000 TB) only sets display state and enforces per-plan upload size limits — it is NOT the purchase flow. The real purchase lives on the Business Model tab: approve USDT → RevenueRouter.processCorporateInvoice() → approve USDT → InayaCorporateEscrow.createEscrow() escrowing the 39% operator COGS share over 12 months.
Staking	Reads InayaStaking for APY/tier display. Stake (with 0/30/90-day lock tier selection, real on-chain lockExpiry), Unstake (blocked until lock expires), Claim reward — direct calls to stake()/withdraw()/claimReward().
My Dashboard	Read-only aggregation of state already fetched elsewhere — PAYG transaction history with BscScan links, Corporate Reserve status, staking overview. No new contract calls.
Referrals (ReferralSection.js)	Deliberately no wallet anywhere — email + Didit KYC only, specifically to detect self-referral. Real KYC gate: referral credit only counts once Didit verification resolves.
Genesis Airdrop	Two mechanics, neither with an on-chain claim function today: an "Upload Reward" progress-bar estimate (0.01 \$INAYA per upload, capped at 0.3/wallet, display-only, not minted), and a "Contributor Allocation" (Community/Developer/Moderator buckets of a 1,000,000-token pool) that routes to Google Forms for manual review — real distribution is off-app at TGE, not automated by this code.

SECTION 08

Business Workspace — SaaS Layer

A separate, structurally distinct product built on the same infrastructure: companies get organizations, departments, projects, and documents with real server-enforced permissions — sold and priced independently of storage/token economics, no wallet literacy required.

- Auth: email magic-link OR Google Sign-In, session cookie based (org.js — createSession/consumeLoginToken), completely separate from the wallet-signed-message pattern used elsewhere.
- Hierarchy: Org → Department → Project → Document, each level with its own role/permission scoping (canManageOrg gates admin actions; VIEW/EDIT/MANAGE per document).
- Workflow: documents move through a real state machine (DRAFT → PENDING → etc.) with an atomic transition function and a persisted activity log — not just a status field that gets overwritten.
- Sharing: explicit per-person grants (document-permissions.js) plus tokenized public share links (hash-and-compare, atomic single-consume where applicable) — same pattern conventions as the Data Room's magic-link/session tokens, implemented independently for each feature.
- Storage model: documents here are encrypted client-side before upload using the same passkey-based flow as the Sovereign Vault, then pinned as JSON-shard blobs — NOT the same storage path as the Data Room, which stores plain, unencrypted PDFs for inline viewing (see Section 11 for why that's a deliberate, different choice).
- Billing: Stripe subscription plans (orgPlans.js), seat limits enforced server-side on invite, storage-size limits enforced server-side on document upload.
- AI: a permission-aware Business Assistant (Section 12) that only ever sees data the requesting user is actually authorized to see.

SECTION 09

Security Layer — Decentralized Threat Intelligence

A separate DePIN-style subsystem: independent nodes report suspicious domains/IPs; the backend reputation-weights reporters and, once confidence crosses a threshold, anchors a CONFIRMED verdict on-chain. Marketed externally as "Inaya Firewall."

1. **Report.** A node (mobile/desktop client or node-daemon's report command) POSTs a signed observation to `/api/security/report`. Rate-limited (200/day/node), upserted per node+threat+day.
2. **Aggregate.** `computeThreatConfidence()` reputation-weights every independent reporter over a 14-day window. Node reputation is real-time off-chain, only checkpointed on-chain periodically — confirmed reports raise it, false positives cost 3x as much (asymmetric penalty by design).
3. **Confirm.** Crossing `CONFIRM_THRESHOLD_BPS` (75%) triggers exactly one relayer-signed `InayaThreatReporter.confirmThreat()` call. A single new/low-reputation node can never force a confirmation alone — verified live in testing: 4 neutral-reputation nodes capped out at 52% confidence, well short of threshold.
4. **Surface.** Public `/security` web page (flagship AI-assistant card, destination checker, live threat feed, Inaya Firewall banner), mobile `SecurityScreen` (Monitor/Protect/Strict modes, local allow/blocklist, network overview), desktop enforcement (Windows `netsh advfirewall` / Linux `iptables block`, IP-literal only — not run against a real machine yet, reviewed line-by-line only), admin dashboard, and a dedicated Security AI Assistant that is explicitly forbidden from inventing evidence — every answer is grounded only in verified `security_*` collection data.

Plaintext indicators (domains/IPs) never touch the chain — threat IDs are keccak256 of the normalized indicator; the backend holds the plaintext↔hash mapping (`src/lib/security.js`).

SECTION 10

Inaya Learn — Educational Platform

A YouTube-based educational discovery layer inside both the web dApp and mobile app — turns Inaya into a daily-use destination beyond storage/staking.

- Search/browse via the real YouTube Data API v3, backend-cached (MongoDB TTL index) specifically because search.list costs 100 quota units/call against a 10,000/day default quota — caching is load-bearing infrastructure here, not an optimization.
- Categories and curated collections (Learn Web3 / Learn AI / Learn Programming) are a hardcoded, git-deployed config file, not an admin-editable database — deliberate V1 scope cut.
- Local-first save/progress (AsyncStorage on mobile, localStorage on web), optional backend sync once a wallet connects.
- An AI Tutor grounded in the opposite guardrail philosophy from the Security Assistant: it teaches using its own general knowledge, only calling tools to check the user's own saved videos/progress — never gated to a narrow data set.

SECTION 11

Investor Data Room

A dedicated, branded document room for sharing investor materials with per-visitor view tracking — replaces sharing a Google Drive folder blind.

- Single shareable link, email required (not optional) + NDA click-through, per-visitor magic-link email verification.
- Deliberately different storage model from the Business Workspace: documents here are plain, unencrypted PDFs (server-proxied streaming, never a raw Pinata URL exposed) so they can be viewed inline with real duration tracking — the Business Workspace's client-side-encrypted-blob model is the wrong fit when the entire point is casual inline viewing with analytics.
- View events (open + 15-second heartbeat + close) roll up into a per-visitor engagement table for the admin — exactly who looked at what, for how long.

SECTION 12

The Three AI Assistants

Business, Security, and Learn Tutor all share one architectural pattern (Gemini function-calling, 5-round tool loop, 429/503 retry with backoff) but deliberately opposite guardrail philosophies.

ASSISTANT	WHERE	GUARDRAIL PHILOSOPHY
Business Assistant	Business Workspace sidebar	Permission-aware — every tool call re-checks the requesting user's actual document/project access; never surfaces data they couldn't otherwise see.
Security Assistant	Public /security page + mobile SecurityScreen	Strict evidence-grounding — explicitly forbidden from inventing or generalizing beyond verified security_* collection data. "Explain phishing in general" is fine; fabricating a specific threat's status is not.
Learn AI Tutor	Web + mobile Learn section	The opposite on purpose — teaches using its own general knowledge like a real tutor; tools exist only to ground it in the user's own saved videos/progress, never to gate what subject matter it can discuss.

SECTION 13

Mobile App

inaya-mobile (Expo/React Native) is a superset of the web dApp's consumer features, plus Business Workspace, Learn, and Security — not a stripped-down companion app.

- Wallet: MetaMask Connect Multichain (direct deeplink to MetaMask Mobile), configured for BNB Chain Testnet specifically.
- Business Workspace: full org/department/project/document hierarchy, permissions, sharing, Bearer-token session auth (separate from web's cookie session), biometric app-lock, Google Sign-In.
- Security: SecurityScreen with protection mode, destination checker, local allow/blocklist, network overview, AI Security Assistant chat.
- Learn: search/browse/watch/save/progress via react-native-youtube-iframe, plus a global AiTutorButton (header icon → full-screen modal) reachable from every Learn screen.
- Watcher Pioneer, Referrals, Faucet-adjacent flows, and Genesis Airdrop all mirror the web dApp's mechanics 1:1 against the same backend routes.

SECTION 14

Desktop Apps

Two separate Tauri (Rust) apps — deliberately thin wrappers, no bundled frontend. Each app's dist/ is a blank placeholder HTML page; the Rust shell just points its webview at the live production URL and layers native OS chrome (tray, menu, notifications, auto-update) on top.

APP	WRAPS	NATIVE ADDITIONS BEYOND THE WEB PAGE
inaya-desktop ("Business Workspace")	inayanetwork.com/business	System tray/minimize-to-tray, native menu, native Windows toast for pending approvals (polls /api/orgs/pending-approvals), signed auto-updater, real firewall-block enforcement (netsh/iptables) for the Security Layer.
inaya-dapp- desktop ("Inaya Network")	inayanetwork.com/	Same tray/menu/updater pattern, no notifications plugin (no equivalent concept on the main dApp). Detects window.__TAURI__ and leads with WalletConnect instead of extension-based wallets, since a native webview has no browser extensions.

Chosen over Electron after measuring both: 8.3MB installed vs 269MB for an equivalent Electron build, since Tauri rides the OS's own WebView2 (Windows) / WebKitGTK (Linux) instead of bundling Chromium. Not code-signed yet (deferred decision, see Section 20) — Windows installs show an expected SmartScreen warning.

SECTION 15

Backend Infrastructure & Conventions

One Next.js app (inaya-network-dapp) serves the web dApp, Business Workspace, every public page, and the entire API surface every client (mobile, both desktop apps) calls.

- Data layer: MongoDB, one lib file per feature domain (security.js, learn.js, dataroom.js, activity.js, orgs.js, watcherPioneer.js, feedback.js...) — each with the same shape: getXCollections(), ensureXIndexes() (module-level idempotency guard), validateXInput() functions that throw (fail-closed).
- Cron jobs (Vercel Cron, vercel.json): settlement-release (midnight), security reputation checkpoint (3am — moved off a 6-hourly schedule after it silently broke every deployment for weeks on the Hobby plan's once-daily cron limit).
- Two admin surfaces: /admin (Enterprise Dashboard — revenue, usage, DAU/WAU, feedback) and /admin/security and /admin/dataroom, all gated by the same ADMIN_DASHBOARD_SECRET ?key= pattern, not a full multi-admin auth system (solo-founder-appropriate, by design).
- Every route follows the same shape: export const dynamic = "force-dynamic", outer try/catch → 500 + console.error, inner try/catch → specific 4xx.

SECTION 16

Identity & Auth Patterns

PATTERN	USED BY	MECHANISM
Signed-message wallet auth	Sovereign Vault sign-up, Security Layer node reports, Watcher Pioneer	ethers.verifyMessage over a reconstructed message string, 5-minute freshness window — reimplemented locally per-feature rather than shared, by established convention in this codebase.
Cookie session (magic-link / Google)	Business Workspace (web)	orgs.js — generateToken/hashToken, TTL-indexed magic_links collection, consumeLoginToken.
Bearer token session	Business Workspace (mobile)	Same session concept as web, token-based instead of cookie-based since mobile has no cookie jar shared with a browser.
Anonymous device/visitor ID	Referrals, Watcher Pioneer, activity pings, Security destination checker, Learn progress (no wallet)	Client-generated UUID cached in localStorage/AsyncStorage, upgraded to wallet address once one connects — same trust model reused across every feature that needs to work pre-wallet.
Investor Data Room session	Data Room only	Its own magic-link + session-cookie pair, deliberately separate from Business Workspace's — different visitor identity model (name+email+NDA, not org membership).

SECTION 17

Activity & Analytics (DAU/WAU)

One small, shared primitive tracks daily/weekly active users across all three surfaces — dApp, Business Workspace, mobile.

- One MongoDB doc per identity+surface+day, upserted (not inserted) — repeated pings the same day are free no-ops, so dedup is structural, not query-time logic.
- DAU = distinct identities pinging today; WAU = distinct identities over the trailing 7 days — both computed with a plain distinct() query since the per-day collapse keeps the working set small.
- Identity: wallet address if connected, otherwise the same anonymous visitor/device ID pattern from Section 16.
- Surfaced as one more section on the existing /admin dashboard — not a new page, not a new auth system.

SECTION 18

End-to-End Flow: Uploading a File

1. User connects wallet, signs a verification message (Sovereign Vault sign-up), sets a master passkey, selects a file.
2. custody-sdk derives an AES key from the passkey (PBKDF2), encrypts the file (AES-GCM-256), and splits the ciphertext into two shards.
3. Each shard is pinned to Pinata/IPFS independently, returning two CIDs.
4. Client checks for a duplicate asset hash, reads live fee rates, approves USDT/\$INAYA if needed, then calls `InayaCustody.batchRegisterAssets()` — this is the transaction that actually makes the upload "real."
5. Client separately calls `InayaProofRegistry.registerMerkleRoot()` per file (a Merkle root over the file's chunks) — a second, independent transaction; if this one fails, the custody registration from step 4 is not rolled back.
6. On download, the reverse: read `InayaCustody.assets(fileHash)` for both CIDs → fetch both shards from Pinata → concatenate → derive the same AES key from the user's passkey → decrypt. The passkey never leaves the device at any point in either direction.

SECTION 19

End-to-End Flow: A Threat Getting Confirmed

1. Independent nodes each observe the same suspicious domain/IP and POST a signed report to `/api/security/report` — rate-limited, deduped per node+threat+day.
2. `computeThreatConfidence()` reputation-weights every independent reporter within a 14-day lookback window.
3. Once aggregate confidence crosses 75%, the backend (acting as an authorized relayer) fires exactly one `InayaThreatReporter.confirmThreat()` call — anchoring category, confidence, and a hash of the contributing-node list on-chain.
4. The public `/security` page, mobile `SecurityScreen`, and desktop enforcement all read the same confirmed-threat feed from that point forward — a confirmed domain gets blocked at the app layer everywhere, and at the OS firewall layer on desktop for a real, confirmed threat.

SECTION 20

Known Gaps & Deliberate Deferrals

Stated plainly, not buried — these are conscious scope cuts, not things quietly forgotten:

- Node-daemon does not store/serve shards or run settlement logic — it's identity + registration + heartbeat only, today.
- InayaEgressTimelockVault has no code path yet that actually pays INAYA/USDT into it — it only accrues balance once something is wired to send it.
- InayaProofRegistry's chunk-proof verification is onlyOwner (backend-checked) rather than permissionless/stake-slashing-backed — a documented future path, not built yet.
- Real Android VPN/packet interception for the Security Layer is out of scope — mobile enforcement checks destinations inside the Inaya app only.
- Desktop firewall-block commands (netsh/iptables) are written and reviewed but have not been executed against a real machine.
- Neither desktop app is code-signed (Authenticode) — investigated, no free option exists for closed-source software, deferred to mainnet rather than spend on it now.
- Genesis Airdrop has no on-chain claim function — both mechanics are display/manual-review only, real distribution happens off-app at TGE.
- macOS is out of scope for both desktop apps, per Apple Developer Program cost + notarization constraint.

SECTION 21

Where Everything Lives

AREA	PATH
Core protocol contracts	contracts/ (repo root) — InayaStaking.sol, InayaEgressTimelockVault.sol, InayaCorporateEscrow.sol, InayaNodeRegistry.sol, InayaProofRegistry.sol, MockINAYA.sol
Security Layer contracts	contracts/InayaThreat{Registry,Reporter}.sol, InayaNodeReputation.sol, InayaSecurityPolicy.sol
Deploy scripts	scripts/deploy.js, deploy-staking.cjs, deploy-vault.cjs, deploy-escrow.cjs, deploy-node-registry.cjs, deploy-verifier-safe.cjs, deploy-security-layer.js
Client-side crypto SDK	inaya-network-dapp/custody-sdk/src/ (crypto.js, index.js, contracts.js)
Node operator CLI	inaya-network-dapp/custody-sdk/packages/node-daemon/
Backend + web dApp + Business Workspace	inaya-network-dapp/src/app/, src/lib/, src/app/api/
Mobile app	inaya-mobile/src/
Desktop — Business Workspace wrapper	inaya-desktop/src-tauri/
Desktop — dApp wrapper	inaya-dapp-desktop/src-tauri/